

Smart Document Parser: Resume Entity Extraction

Project Report (Assignment 3)

GitHub Repo: <https://github.com/Akshat-A-K/Resumer-Parser>

Demo video: [video](#)

Team: Akshat Kotadia (2025201005, akshat.kotadia@students.iiit.ac.in)

Om Mehra (2025201008, om.mehra@students.iiit.ac.in)

Ankit Chavda (2025201045, chavda.ankitbhai@students.iiit.ac.in)

1. Abstract

We built a resume parser that converts PDF or image resumes into clean JSON. The system uses OCR and PDF text extraction, and a local LLM (Ollama with Llama 3.2) to extract entities like name, email, skills, education, and experience. A Streamlit web app lets the user upload a file, review the output, edit it, and download the final JSON. We also created an evaluation mode that compares predictions with a labeled dataset and reports precision, recall, and F1 scores. This report explains the problem, dataset, method, and results for two approaches. The code is in the branches: `ollama-basic` and `prompt-based-fine-tuning`. Both branches are considered in this submission.

2. Introduction

In hiring, companies receive many resumes for one job opening. Manually reading each resume and writing the details is slow. So we need an automated system that can read a resume and extract important data in a structured format. The assignment asks for a smart document parser, and we selected the resume parsing task. Our system is fully local and works without paid APIs.

3. Problem Statement and Objectives

The system should take a resume in PDF or image form and return a structured JSON with key information. It should handle different layouts and noisy text and still give usable results.

We allow both PDF resumes and image resumes (PNG, JPG, JPEG, BMP, TIFF, WEBP). For images we use OCR, and for PDFs we extract text directly.

Main objectives:

- Extract contact, education, experience, skills, and projects.
- Provide a simple UI for upload, review, edit, and download.
- Provide evaluation results on labeled data.

4. Dataset

We used the Resume Entities for NER dataset from Kaggle [1]. It has 220 annotated resume samples in newline-delimited JSON format. Each sample has resume text and labeled entity spans. The labeled fields include Name, Email Address, Location, Designation, Companies worked at, Skills, College Name, Degree, Graduation Year, and Years of Experience. Many fields in our schema (such as phone, links, projects, or certifications) do not have ground truth in the dataset, so evaluation is only for the mapped 10 fields.

5. Approach 1: Streamlit Application (ollama-basic)

This approach is in the branch `ollama-basic`. The pipeline is:

1. User uploads a PDF or image.
2. Text is extracted using PDF parsers for PDF, and OCR for images.
3. A strict JSON schema prompt is sent to the local LLM.
4. The output is cleaned and validated into the schema.
5. Results are shown in the UI, user can edit, and then download JSON or CSV.

How the Streamlit app looks and how it is used for ML work: the interface is simple and clean, with a left sidebar to choose the mode (Parse One Resume or Evaluate Quality). In the main area, the user sees the file upload, the extracted text preview, and the editable output. For ML work, it is useful because it gives fast feedback, lets us correct output, and makes it easy to test many samples without extra scripts.

Key points of this approach:

- Uses local Ollama with Llama 3.2 model and temperature 0.
- Uses a single prompt with selected fields.
- Results are validated so empty or missing values are handled safely.
- Evaluation mode runs on a chosen number of labeled samples.

6. Models and Tools Used

- Streamlit for the UI [2].
- PyMuPDF with pypdf fallback for PDF text extraction [5].
- EasyOCR with Pillow and NumPy for image text extraction [7].
- Ollama for local LLM inference (Llama 3.2, model must be downloaded) [3, 4].
- Pydantic for schema validation and normalization [6].
- Pandas for evaluation tables.

7. Approach 2: Prompt-Based Fine-Tuning (prompt-based-fine-tuning)

This approach is in the branch prompt-based-fine-tuning. It focuses on stronger prompt engineering and structured extraction. We include it here for comparison.

Key ideas:

- Section-wise prompting: resume text is split into sections like education, experience, projects, and skills. Each section is parsed with only relevant fields.
- Extended schema: structured lists are used for education, experience, and projects.
- Extra normalization: JSON repair, retries, and link mapping to improve output quality.
- Caching: repeated extractions are faster.

This method aims to improve list-heavy fields by reducing prompt noise and focusing on relevant text for each section.

8. Branch Comparison and Key Differences

- prompt-based-fine-tuning has section-wise prompting and stronger JSON repair with retries.
- It adds a more detailed schema with structured lists for education, experience, and projects.
- It uses caching to speed up repeated extractions.
- ollama-basic is simpler and faster, with a single prompt for selected fields.
- Evaluation scripts and results are organized a bit differently in the two branches.

9. Evaluation Methodology

We used the labeled dataset to measure accuracy. For string fields, we compute exact match, fuzzy match, and token-based precision/recall/F1. For list fields, we use set-based precision/recall/F1 with fuzzy matching and Jaccard similarity. We also report macro-averaged precision, recall, and F1 across all fields.

10. Results for Streamlit Approach (10 Samples)

Latest evaluation for the ollama-basic branch was run on 10 samples. Macro scores:

- Precision: 0.5712
- Recall: 0.5883
- F1: 0.5561

Entity Type	Precision	Recall	F1	Support
Name	0.90	0.90	0.90	9
Graduation Year	0.40	0.40	0.40	7
Years of Experience	0.50	0.50	0.50	2
Email	0.70	0.60	0.6333	8
Location	0.45	0.80	0.5633	8
Degree	0.80	0.625	0.6857	8
Designation	0.70	0.65	0.6667	8
Companies Worked At	0.2417	0.50	0.3067	8
Skills	0.15	0.20	0.1667	10
College Name	0.87	0.7083	0.7389	10

11. Results for Prompt-Based Fine-Tuning Branch (10 Samples)

Reported macro scores from the prompt-based-fine-tuning branch:

- Precision: 0.585
- Recall: 0.617
- F1: 0.575

Entity Type	Precision	Recall	F1	Support
Name	0.900	0.900	0.900	9
Email	0.700	0.600	0.633	8
Location	0.500	0.900	0.630	8
Graduation Year	0.400	0.400	0.400	7
Years of Experience	0.600	0.550	0.566	2
College Name	0.783	0.725	0.745	10
Degree	0.800	0.650	0.700	8
Designation	0.600	0.583	0.550	8
Companies Worked At	0.541	0.766	0.586	8
Skills	0.028	0.100	0.044	10

12. Comparison and Discussion

Macro-average comparison (10 samples each):

Approach	Precision	Recall	F1
ollama-basic	0.5712	0.5883	0.5561
prompt-based-fine-tuning	0.5850	0.6170	0.5750

Observations in simple words:

- Both approaches are strong in name and education fields.
- The prompt-based branch gives slightly higher macro scores.
- Prompt-based method takes more time because it runs more steps and more prompts.
- Skills and companies are the hardest fields in both approaches.
- Section-wise prompting helps in some list fields, but skills are still weak because of different naming styles.

13. Limitations

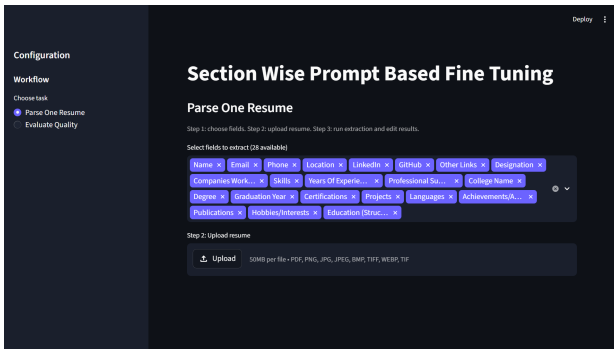
- The evaluation used only 10 samples, so results can change with more data.
- OCR quality depends on resume image quality.
- Some fields are not in the dataset, so they are not evaluated.

14. Conclusion and Future Work

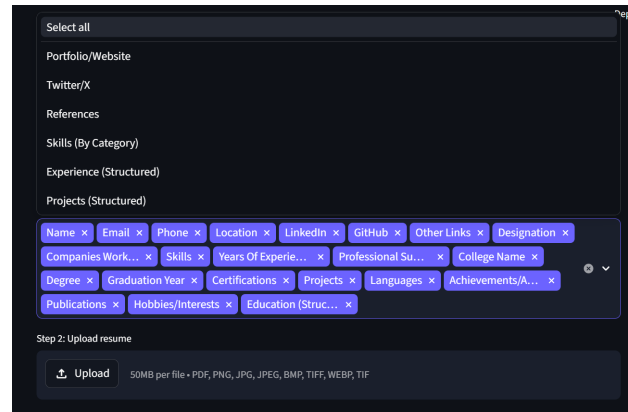
We built a working resume parser with a Streamlit app, and we also tested a prompt-based fine-tuning approach. The ollama-basic approach is simpler and stable. The prompt-based-fine-tuning approach gives slightly better macro scores on the same sample size, but takes more time. For future work, we should run larger evaluations, improve skill normalization, and try a larger model for better accuracy.

15. Screenshots

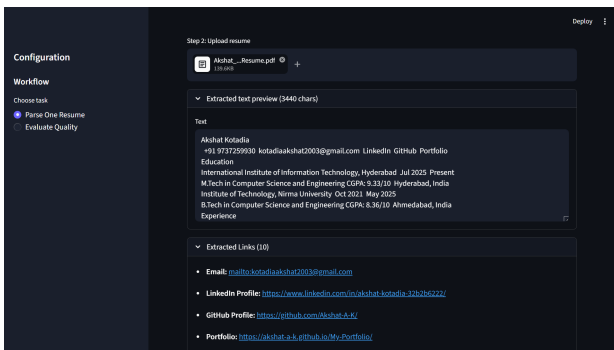
All screenshots below are from the prompt-based fine-tuning implementation.



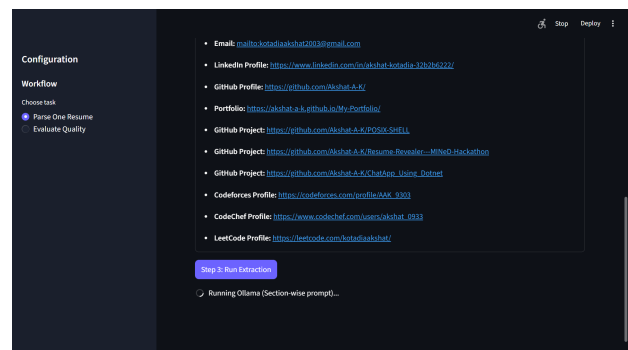
Parse One Resume: field selection and upload



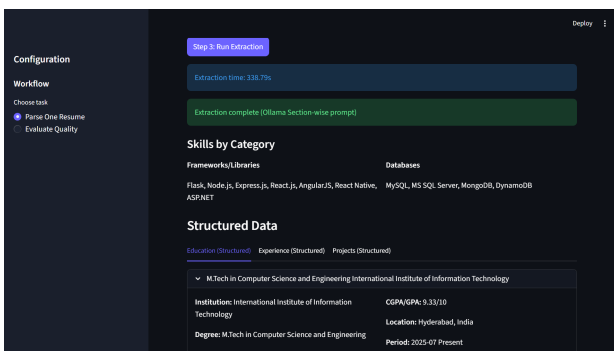
Parse One Resume: field selector dropdown



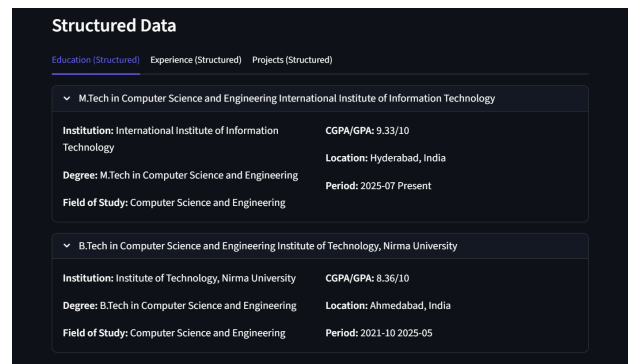
Parse One Resume: extracted text and links preview



Parse One Resume: run extraction and status



Extraction complete: skills by category and tabs



Structured Data: education tab

Structured Data

Education (Structured) [Experience \(Structured\)](#) Projects (Structured)

▼ Software Developer Intern @ Iolite Softwares Pvt. Ltd.

Company: Iolite Softwares Pvt. Ltd.

Period: Jan 2025 – Jun 2025 None

Role: Software Developer Intern

Tech Stack: AngularJS, .NET 9, C#, MS SQL Server

Location: Ahmedabad, India

Description: Worked on enterprise-grade IP Management System using AngularJS (frontend) and .NET 9 (backend). Developed APIs, integrated business logic, and optimized performance.

▼ Software Developer Intern @ Solvemate Tech Pte Ltd.

Company: Solvemate Tech Pte Ltd.

Period: May 2024 – Jul 2024 None

Role: Software Developer Intern

Tech Stack: HTML, CSS, JS, React, Node.js, React Native, AWS, Google Analytics

Location: Ahmedabad, India

Description: Integrated OAuth2.0-based Google Auth with Analytics in a MERN dashboard. Developed "Confession" team-building feature using AWS API Gateway, DynamoDB, React. Built a React Native loyalty app with referral rewards using serverless AWS APIs.

Structured Data		
Education (Structured)	Experience (Structured)	Projects (Structured)
▼ POSIX Shell Implementation		
Name: POSIX Shell Implementation		Link: GitHub Project
Tech Stack: C++		Date: 2025-08
Description: Built a Unix-like shell in C++ with support for foreground/background processes, I/O redirection, and pipelines.		
▼ Resume Revealer		
Name: Resume Revealer		Link: GitHub Project
Tech Stack: Python, Machine Learning, NLP		Date: 2024-02
Description: Created a resume parser that extracts and classifies sections like education, experience, and skills from various formats.		
▼ Chat App		
Name: Chat App		Link: GitHub
Tech Stack: C#, MongoDB, ASP.NET		Date: 2024-11
Description: Developed a real time chat application with features like Register, Login, Logout, and Chat, using ASP.NET Core.		

Deploy

Configuration

Workflow

Choose task

☒ Parse One Resume

Evaluate Quality

Editable Output

Name

Alshah Kotadia

Press Enter to submit form

Email

totadiaseelshah2301@gmail.com

Phone

+91 9737259930

Location

LinkedIn

<https://www.linkedin.com/in/alshah-kotadia-320264222/>

GitHub

<https://github.com/akshat-A-K/>

Other Links

<https://leetcode.com/alshahakshah/>, https://codeforces.com/profile/AK_3003, https://www.codetf.com/users/akshat_0033, <mailto:totadiaseelshah2301@gmail.com>, <https://www.linkedin.com/in/alshah-kotadia-320264222/>, <https://github.com/akshat-A-K/>, https://www.codingf.com/users/akshat_0033

Designation

Software Developer Intern, Software Developer Intern

Companies Worked At

IoLite Softwares Pvt. Ltd., Solvemate Tech Pte Ltd.

Skills

C, C++, Python, Java, JavaScript, C#

Years Of Experience

Professional Summary

College Name

International Institute of Information Technology, Institute of Technology, Nirma University

Degree

M.Tech in Computer Science and Engineering, B.Tech in Computer Science and Engineering

Degree

M.Tech in Computer Science and Engineering, B.Tech in Computer Science and Engineering

Graduation Year

Present

Certifications

Projects

POSIX Shell Implementation, Resume Revealer, Chat App

Languages

C, Python, JavaScript, Java

Achievements/Awards

{ 'CategoryName': 'GATE CS/IT', 'Score': 752, 'Rank': 665, 'CategoryName': 'PGEE', 'Score': None, 'Rank': 101, 'CategoryName': 'MIND 2024', 'Score': None, 'Rank': 'First Runner-Up 1', 'CategoryName': 'Junior Student Placement Coordinator', 'Description': 'Coordinated and mentored 100+ students in their placements, achieved a 95% success rate in placements.', 'CategoryName': 'Mentorship Award', 'Score': None, 'Rank': 'Mentor of the Year' }

Publications

Hobbies/Interests

[{"CategoryName": "Competitive Programming", "Description": "Codeforces 1468 (Specialist), CodeChef 1785 (3 star)", "CategoryName": "LeetCode", "Description": "leetcodeakshat"}]

Portfolio/Website

https://akshat-s-k.github.io/My-Portfolio/

Twitter/X

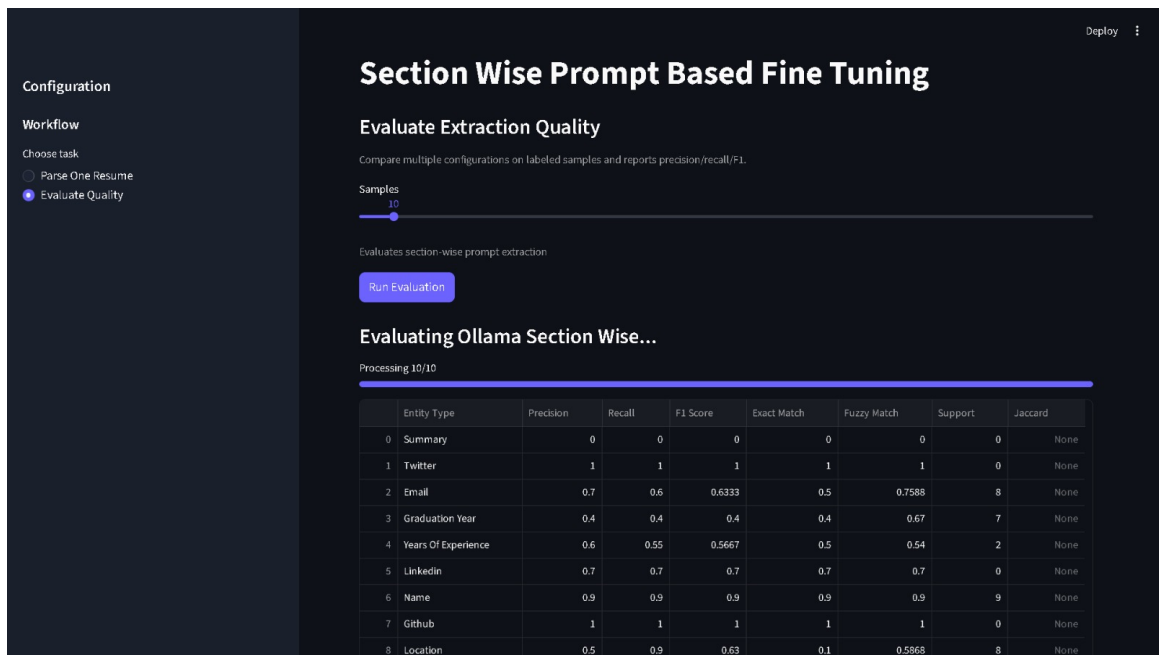
References

Skills (By Category)

[{"Frameworks/Libraries": ["Flash", "Node.js", "Express.js", "React.js", "AngularJS", "React Native", "ASP.NET"], "Databases": ["MySQL", "MS SQL Se

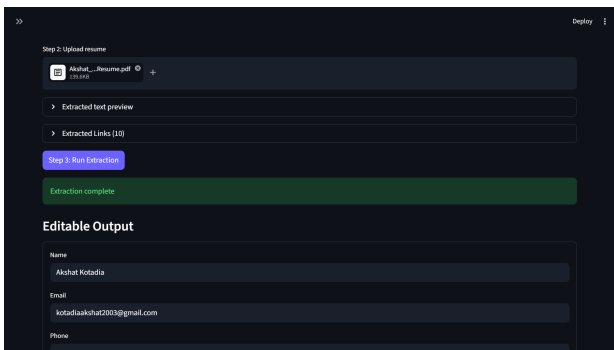
[illegible]

The screenshot shows the Ollama web interface. On the left is a dark sidebar with a 'Configuration' section containing 'Workflow' and 'Choose task'. Under 'Choose task', there are three options: 'Partial One Resume', 'Evaluate Quality' (which is selected with a blue dot), and 'Evaluate Quality'. The main area has a light blue header with a settings icon, 'Stop', and 'Deploy' buttons. The main content area has a title 'Section Wise Prompt Based Fine Tuning' and a subtitle 'Evaluate Extraction Quality'. Below this is a description: 'Compare multiple configurations on labeled samples and reports precision/recall/F1.' There is a 'Samples' input field with a blue arrow icon. Below that is a label 'Evaluates section-wise prompt extraction' and a blue button labeled 'Run Evaluation'. At the bottom, there is a section titled 'Evaluating Ollama Section Wise...' with a progress bar and the text 'Processing L15'.

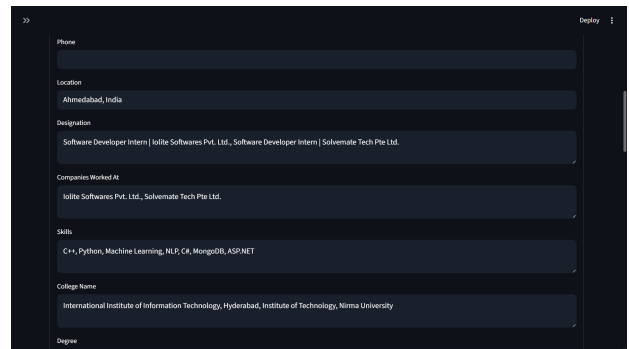


Evaluate Quality: results table

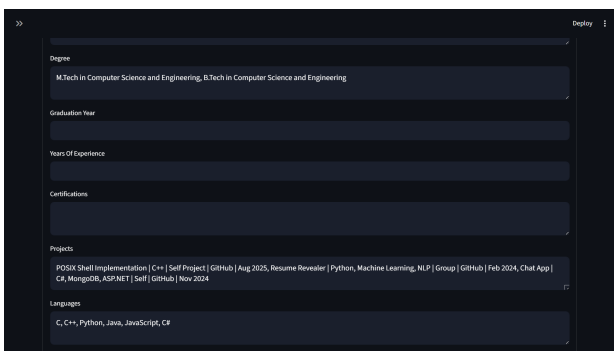
Basic model (ollama-basic) screenshots:



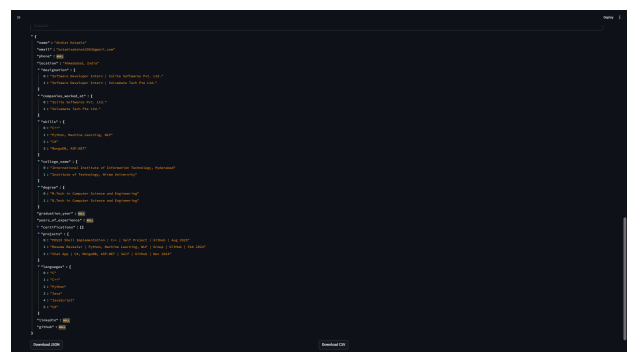
Basic model: upload resume and run extraction



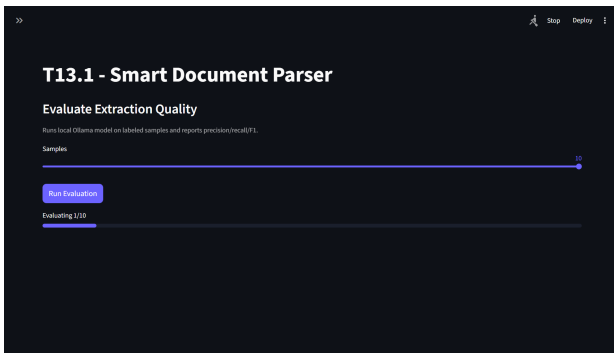
Basic model: editable output (contact, designation, skills)



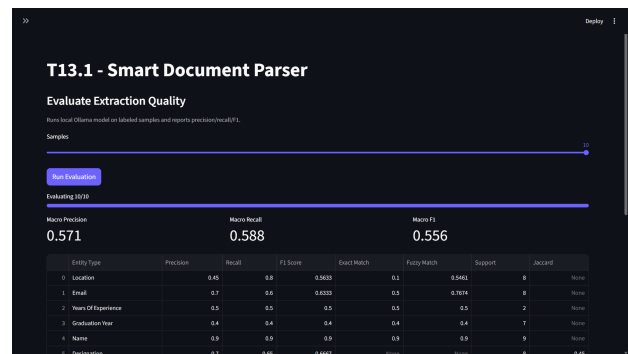
Basic model: editable output (education, projects, languages)



Basic model: final JSON output and downloads



Basic model: evaluation setup and progress



Basic model: evaluation results and metrics table

References

- [1] Dataturks. Resume Entities for NER. Kaggle, 2019.
<https://kaggle.com/datasets/dataturks/resume-entities-for-ner>
- [2] Streamlit Team. Streamlit Documentation.
<https://docs.streamlit.io>
- [3] Ollama Team. Ollama Documentation.
<https://ollama.ai>
- [4] Meta AI. Llama 3.2 Model Card.
<https://ai.meta.com/llama/>
- [5] Artifex Software. PyMuPDF Documentation.
<https://pymupdf.readthedocs.io>
- [6] Samuel Colvin et al. Pydantic Documentation.
<https://docs.pydantic.dev>
- [7] JaiedAI. EasyOCR: Ready-to-Use OCR with Deep Learning. GitHub Repository.
<https://github.com/JaiedAI/EasyOCR>